UNIVERSITEIT VAN PRETORIA
UNIVERSITY OF PRETORIA
YUNIBESITHI YA PRETORIA
Denkleiers • Leading Minds • Dikgopolo tša Dihlalefi

COS 216 Practical Assignment 4

- Date Issued: **7 April 2025**
- Date Due: **12 May 2025** before **11:00**
- Submission Procedure: **Upload to the web server (`wheatley`) + ClickUP**
- This assignment consists of **7 tasks** for a total of **145 marks**.

1 Introduction

During this practical, you will be completing the functionality of your e-commerce website and accompanying API. After completing this assignment, your website will make requests to the API you created in the previous practical. A user will be able to view products, add them to their wishlist and add them to their cart — but now, the API will return persistent data stored in your database. The user will also be able to order products on their cart.

The specific goal for this assignment is to showcase the following functionality:

- Implementing the "save", "wishlist", "cart" and "order" types of the API.
- The ability to login and logout.
- The ability to set and save user preferences.
- The ability to add/remove products from your wishlist.
- The ability to add/remove products from your cart.
- The ability to order products from your cart.

2 Constraints

1. You must complete this assignment individually.
2. You may ask the Teaching Assistants for help but they will not be able to give you the solutions.
3. You must produce all of the source files yourself; you may not use any tool to generate source files or fragments thereof automatically **(This includes ChatGPT, Gemini etc.!!).**
4. Your assignment will be viewed using Brave Web Browser (`https://brave.com/`) so be sure to test your assignment in this browser. Nevertheless, you should take care to follow published standards and make sure that your assignment works in as many browsers as possible.
5. You may utilise any text editor or IDE, upon an OS of your choice, again, as long as you do not make use of any tools to generate your assignment. **(This includes ChatGPT, Gemini etc.!!)**
6. All written code should contain comments including your name, surname and student number at the top of each file.
7. Your assignment must work on the `wheatley` web server, as you will be marked off there.
8. **You may not use external libraries to perform security operations (You may use PHP built-in functionality).**
9. **Server-side scripting should be done using an Object Oriented approach.**

3 Submission Instructions

You are required to upload all your source files (e.g. HTML5 documents, any images, etc.) to the web server (`wheatley`) and clickUP in a compressed (zip) archive. Make sure that you test your submission to the web server thoroughly. All the menu items, links, buttons, *etc.* must work and all your images must load. Make sure that your practical assignment works on the web server before the deadline. No late submissions will be accepted, so make sure you upload in good time. The server will not be accepting any uploads and updates to files from the stipulated deadline time until the end of the marking week (Thursday at 3pm).

The deadline is on Sunday but we will allow you to upload until Monday 11am. After this NO more submissions will be accepted.

Note, `wheatley` is currently available from anywhere. But do not rely that outside access from the UP network will always work as intended. You must therefore make sure that you `ftp` your assignment to the web server. Also make sure that you do this in good time. A snapshot of the web server will be taken just after the submission was due and only files in the snapshot will be marked.

Practicals are marked by demonstrating your practical to a tutor during the allotted marking weeks. If you do not demonstrate your practical you will receive 0 for the practical. You will only be marked in the practical session that you have booked on the cs portal, if you miss it, you will receive 0 (Unless special permissions have been granted by the lecturer. i.e. You were sick and able to provide a sick note).

NB: You must also submit a ReadMe.txt file.
It should detail the following:

- how to use your website
- default login details (username and password) for a user you have on your API
- any functionality not implemented
- bonus features that you have implemented

4 Online resources

PHP Sessions - `http://www.w3schools.com/php/php_sessions.asp`

PHPMyAdmin - `http://www.phpmyadmin.net/home_page/index.php`

Timestamps - `https://en.wikipedia.org/wiki/Unix_time`

Cookie - `https://www.w3schools.com/js/js_cookies.asp`, `https://developer.mozilla.org/en-US/docs/Web/API/Document/cookie`

Local Storage - `https://www.w3schools.com/jsref/prop_win_localstorage.asp`

5 Rubric for marking

PHP API	
Works off PHP API	5
Login & Logout	
HTML	5
Security	5
API + Validation	5
Cookie/Local DOM Storage	
Theme	10
API Key	3
JS	12
`save` **API type**	
API + MYSQL	10
Filtering	5
Authorization + Validation	10
`wishlist` **API type**	
API + MYSQL	10
Displayed on wishlist page	5
Authorization + Validation	5
`cart` **API type**	
API + MYSQL	10
Displayed on cart page	5
Authorization + Validation	5
`order` **API type**	
API + MYSQL	20
Displayed on website	10
Authorization + Validation	5
Upload	
Does not work on `wheatley`	**-30**
Not uploaded to clickUP	**-145**
Not demoed	**-145**
Bonus	10
Total	**145**

6 Assignment Instructions

Task 1: Integrating your PHP API ..... (5 marks)

For this task, you should alter your **Products** and **Deals** pages so that they makes use of the PHP API you created in PA3, instead of the Wheatley API used in PA2. The filter/search/sort should now work using your PHP API. Since the PA3 version of the API does not have all of the *type* parameters of the Wheatley API, you may continue using "GetDistinct" to retrieve the filter options. You should also retain the frontend functionality for currency conversion that you previously implemented in PA2.

Task 2: Login & Logout ..... (15 marks)

This task requires you to implement login and logout functionality for your website. Implement the login API type for your PHP API. You should return the API key of the user once they logged in successfully. Your API should follow the following structure:

Request
url: wheatley.cs.up.ac.za/uXXXXXXXX/api.php - (URL to your PHP API)
method: POST - (HTTP Method)

JSON POST body

Parameter	Required	Description
type	Required	Method to identify what information is needed, consists of the following values: • ***Login*** - Tells the API what functionality to perform
email	Required	The email of the user registering
password	Required	The users password

Request Example Here is an example of the post parameters you can use.
Example Post body:

```
{
        "type":"Login",
        "email": "tony@starkindustries.com",
        "password": "LoveYou3000"
}
```

Response Object
Your API should return a structured JSON Object as a response. Here is what the response to the request example given above should be:

```
{
        "status": "success",
        "timestamp":"1679507636541",
        "data": [
            {
                "apikey": "5c331d9c15d564d3d0de0f1f2937b92e"
             }
        ]
}
```

After implementing the login functionality in the API, make a basic login page in `login.php`.

NB: Once a user has successfully logged in, their API key needs to be returned, and any API requests need to use this API key (for retrieving product information and user settings). You will need to store this in a cookie or local DOM storage as seen in Task 3. Ensure that your API only accepts valid requests.

The second part of this task requires you to implement logout functionality which simply clears and removes the cookie/session. (Add this functionality to `logout.php`)
Remember, that the logout option should only be displayed to users that are logged in.

Task 3: Cookie or Local DOM Storage ..... (25 marks)

Storing the API key in the cookie or local DOM storage makes it easier to retrieve the key to make the requests to your PHP API. Once a user has logged in, you should create either a cookie or local DOM storage with the API key that was returned during the login process.

You will also use the cookie or local DOM storage for some CSS styling preferences. These preferences must be saved and remembered each time a user loads your site. You may also include this as a User preference and sync to your database. Many websites have a themed CSS styling, where a user is allowed to choose a theme (light or dark). You need to implement this and at least have functionality to change between themes in the footer of the webpage. You **MUST** have a light and dark theme. When a theme is changed, it should dynamically be updated (the user should not have to reload the page).
In addition, a user should have default filters applied that they can save using the 'save' API type explained in the next task.

Task 4: `save` PHP API type ..... (25 marks)

For this task, you will need to implement the `"save"` type for the API. This feature simply allows a user to change their preferences. These preferences are the **filters** you used in previous practicals for the "Products" page, as well as the **default currency** for products. You are only required to save the filters (such as "Department", "country_of_origin") and preferred currency, but you are welcome to save search or sort as well. You can set the default (preferred) currency to "ZAR" and then update it in the database whenever the user changes the currency on your website.

Note: You should choose your own way of doing this, however remember that **only registered users** can update their preferences.

Once these are saved, they should reflect on the "Products" page, by already having these filters applied (based on the API key). For example, if the user chooses their preference for "Price" to only display products between a minimum and a maximum, then the "Products" page should only show products in that price range. You will need to restructure your database for this. Make sure that the correct user's preferences are updated.

In order to display this functionality, you will need to either create a settings page or add a button to the 'Products' page to save preferences based on the current filters/currency. The user's preferred currency must reflect across your website.

Hint: There are multiple ways to achieve this, you can use session variables to set the preferences or make your API return the preferences, on login and store it in DOM storage.

Task 5: `wishlist` PHP API type ..... (20 marks)

For this task, you will need to implement the "wishlist" type for the API. This task consist of two parts:

- Adding to wishlist:
  When a product is added to a user's wishlist, it should update your database and be displayed on the "Wishlist" page.
- Removing from wishlist:
  When a product is removed from a user's wishlist, it should update your database and should not be displayed on the "Wishlist" page anymore.

From PA1, you should already have a way of adding and removing products in your Wishlist (such as a heart icon in your HTML).

You will also need to modify your database design. You can create a new database table with the wishlisted products for each user.

Below is the schema for the "wishlist" table:

`wishlists`
This table will simply store the products that the user has added to their wishlist.

- temp_id (Primary key)
- customer_id (Foreign key)
- product_id (Foreign key)
- createdAt (Date when product was added to cart)

Additionally, please note the following:

- You should add a "wishlist" API type to your PHP API. It is up to you if you want to handle the "remove from wishlist" through the same type or create a new one.
- **Only registered users can "add to wishlist".** Therefore the API key must be a required parameter.
- The user's wishlisted products should be displayed on the "Wishlist" page. Once a user logs in and navigates to Wishlist, they should see only the products they added to their wishlist.

Task 6: `cart` and `order` PHP API type ..... (55 marks)

For this task, you will need to implement the "cart" type for the API. This task consist of three parts:

- Adding to cart:
  When a product is added to a user's cart, it should update your database and be displayed on the "cart" page.
- Removing from cart:
  When a product is removed from a user's cart, it should update your database and should not be displayed on the "cart" page anymore.
- Placing an order:
  When the user would like to order the products in their cart, an order must be created to track their incoming delivery. (For this practical, you will not focus on details related the actual delivery.)

Similar to your 'wishlist' page, you should already have a way of adding and removing products in your cart (such as a cart icon to add and bin icon to remove).

You will need to modify your database design to cater for the cart, as well as orders for products. You will need to create three tables called `carts`, `orders` and `orders_products` respectively (remember to prefix all table names with your student number).

Below are the schemas for the tables:

`carts`
This table will simply store the products that the user has added to their cart.

- temp_id (Primary key)
- customer_id (Foreign key)
- product_id (Foreign key)
- createdAt (Date when product was added to cart)
- quantity

`orders`
This table will store the orders made by a customer, and track details about them.

- order_id (Primary key)
- customer_id (Foreign key)
- state (A string between the following options: **"Storage", "Dispatched" or "Delivered"**)
- delivery_date (Make this any date between the 19th and 25th of May)
- createdAt (Date when order was made)

`order_products`
This is a join table to represent the many-to-many relationship between orders and their products. (One order can have many products, each with different quantities)

- id (primary key)
- order_id (foreign key)
- product_id (foreign key)
- quantity

Additionally, please note the following:

- You should add a "cart" API type to your PHP API. It is up to you if you want to handle the "remove from cart" through the same type or create a new one.
- You should also add an "order" API type to your PHP API.
- Once a user creates an order, their cart must be emptied.
- **Only registered users can "add to cart" or create an order.** Therefore the API key must be a required parameter.
- The user's wishlisted products should be displayed on the "Wishlist" page. Once a user logs in and navigates to Wishlist, they should see only the products they added to their wishlist.
- You must provide the user a way to view their orders.

Task 7: Bonus ..... (10 marks)

You may add additional 'nice to have' features and depending on the level of difficulty marks will be given.